PRACTICAL REPORT

Configure Infrastructure and Automate Application Using Ansible

Lab 3: Monitoring, Logging & DevSecOps — Experiment 1

Objective	Configure infrastructure and automate application configuration using Ansible playbooks.
Environment	Windows with WSL 2, Ubuntu, Ansible Core 2.20.1, and Nginx.
Target	localhost (local execution in Ubuntu/WSL).

Prepared from practical execution screenshots and verified terminal output.

1. Aim

To configure infrastructure and automate application configuration using Ansible by writing a playbook and applying it to a target node.

2. Tools and Requirements

- Windows with WSL 2 and Ubuntu
- Ansible Core 2.20.1
- YAML playbook and inventory file
- Nginx web server
- Terminal commands for validation

3. Procedure

Step 1 — Open Ubuntu and update packages

Launch Ubuntu through WSL and run:

```
sudo apt update
```

Step 2 — Install and verify Ansible

Install Ansible using the command below, then verify the installation.

```
sudo apt install ansible -y
ansible --version
```

Step 3 — Create the project and inventory

Create the project directory ~/ansible-practical and save the following as inventory.ini:

```
[webservers]
localhost ansible_connection=local
```

Step 4 — Test connectivity

Run the command below. The test returned SUCCESS and pong.

```
ansible -i inventory.ini webservers -m ping
```

Step 5 — Create the playbook

Save the following YAML as setup-webserver.yml:

```
---
- name: Configure infrastructure and automate application
  hosts: webservers
  become: true

  tasks:
    - name: Update package cache
      ansible.builtin.apt:
        update_cache: true

    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present

    - name: Create sample web page
      ansible.builtin.copy:
```

```
    content: |
      <html>
      <body>
      <h1>Ansible Automation Successful!</h1>
      <p>Application configured using Ansible.</p>
      </body>
      </html>
    dest: /var/www/html/index.html
    mode: "0644"

- name: Start and enable Nginx
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```

4. Validation and Execution

Syntax validation

Run the syntax check before execution:

```
ansible-playbook -i inventory.ini setup-webserver.yml --syntax-check
```

The syntax check passed and displayed the playbook filename.

Playbook execution

```
ansible-playbook -i inventory.ini setup-webserver.yml
```

During troubleshooting, a temporary sudoers rule was used to resolve the interactive privilege-escalation timeout. The temporary rule was removed after the successful run.

Application verification

```
curl http://localhost
```

The command returned the deployed HTML page, including the heading “Ansible Automation Successful!”.

5. Result

The playbook completed successfully. The recorded recap was ok=5, changed=2, unreachable=0, failed=0. Nginx was installed and started, and the sample web page was returned by the local HTTP request.

6. Screenshots / Evidence

Evidence 1

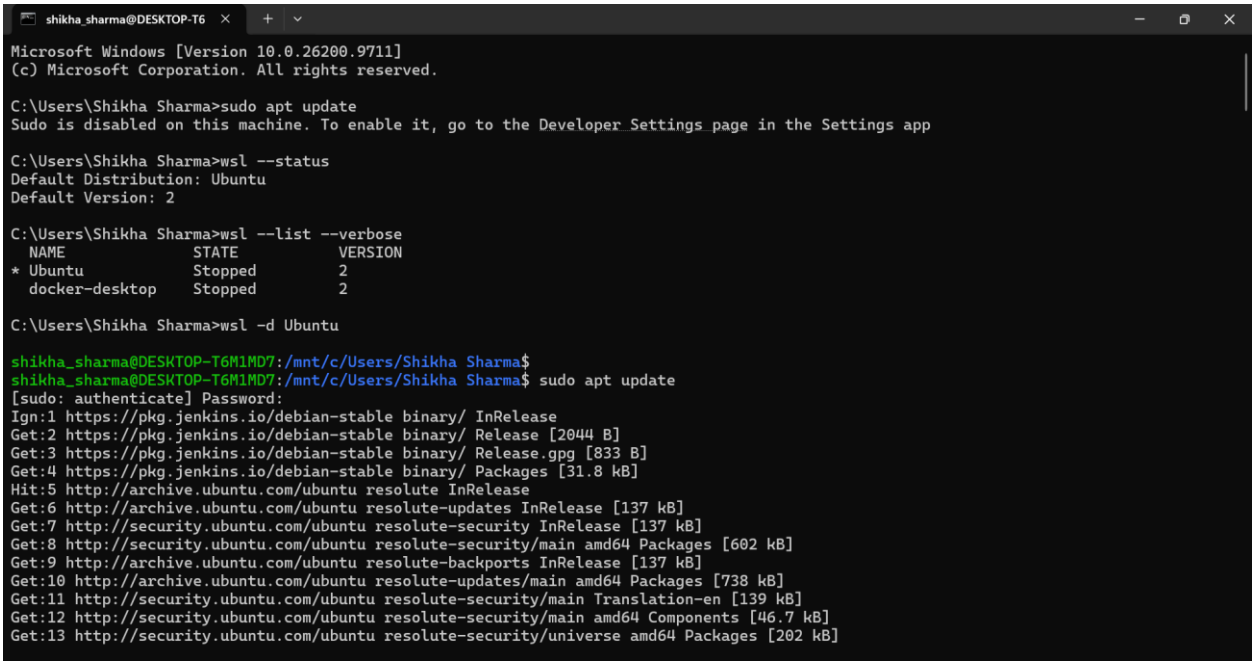


Figure 1. Opening Ubuntu in WSL and updating the package index.

Evidence 2

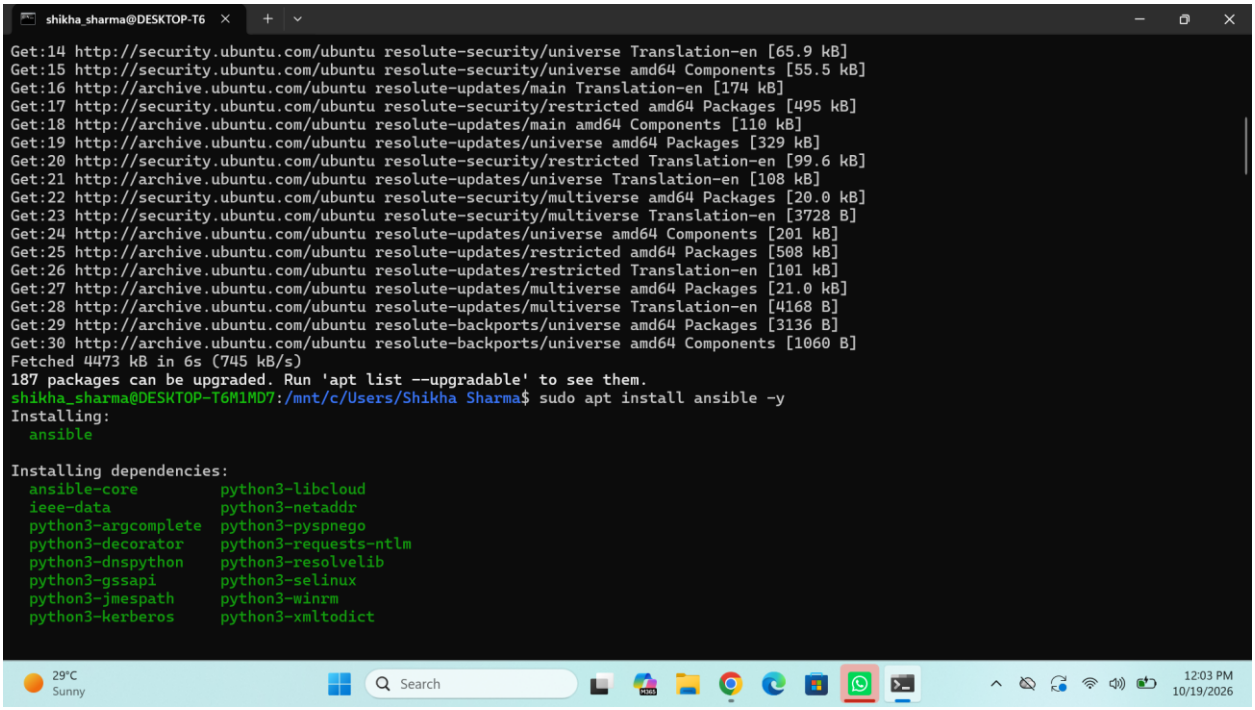


Figure 2. Installing Ansible and its required dependencies.

Evidence 3

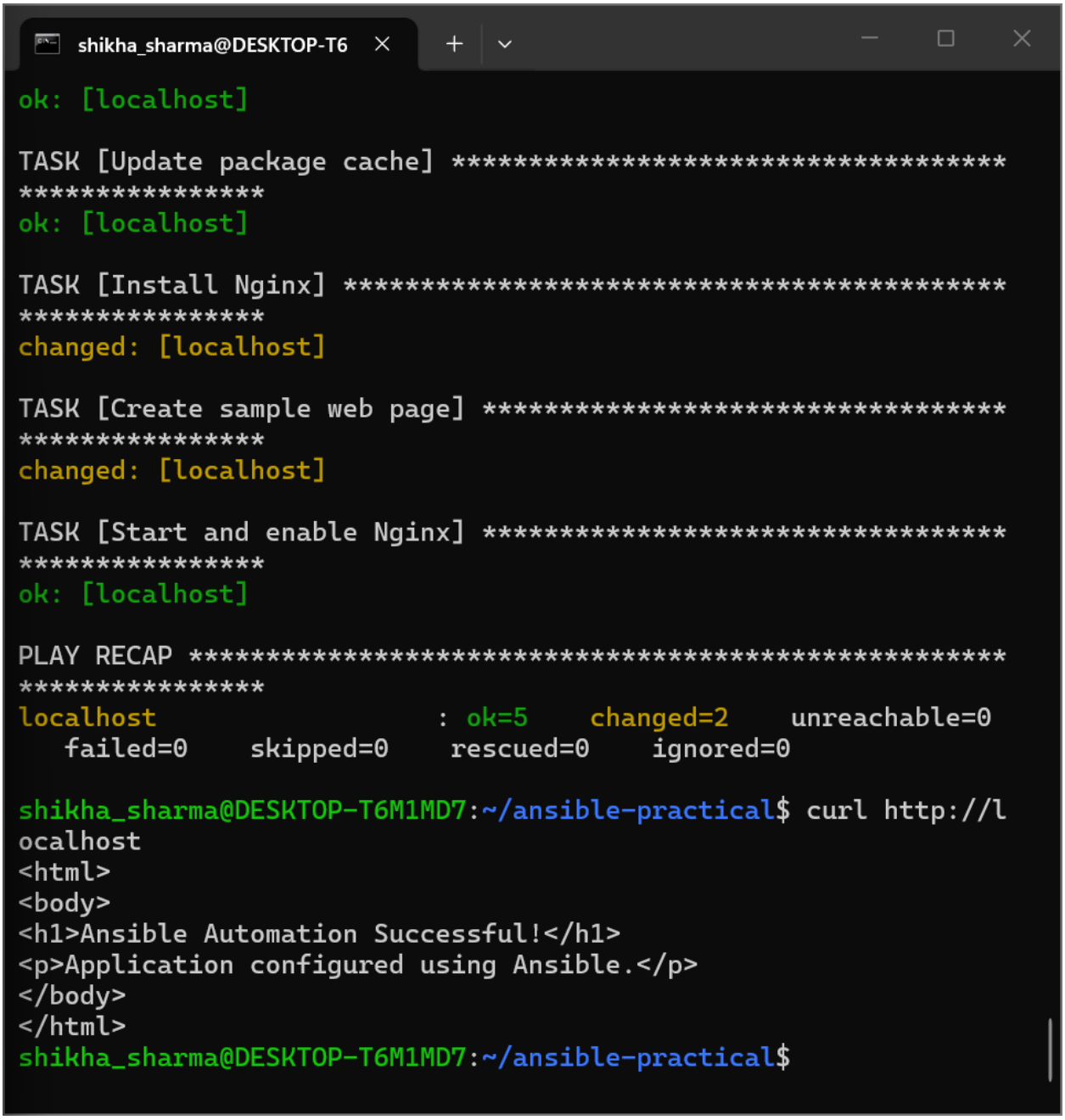


Figure 3. Successful playbook execution and application verification.

7. Conclusion

This experiment demonstrated local infrastructure and application automation with Ansible. The inventory identified localhost as the target, the playbook installed and configured Nginx, and the deployment was verified using curl. The playbook finished with zero failed or unreachable tasks.

Note: This execution used localhost as the target node. If the lab evaluator requires a separate remote target node, update the inventory with that node's SSH details and run the playbook against it.